

Dash Basics: HTML and Core Components



Objectives

After completing the lab you will be able to:

- Create a dash application layout
- Add HTML H1, P, and Div components
- Add core graph component
- Add multiple charts

Estimated time needed: 30 minutes

Dataset Used

[Airline Reporting Carrier On-Time Performance](#)

About Skills Network Cloud IDE

This Skills Network Labs Cloud IDE (Integrated Development Environment) provides a hands-on environment in your web browser for completing course and project related labs. It utilizes Theia, an open-source IDE platform, that can be run on desktop or on the cloud. So far in the course you have been using Jupyter notebooks to run your python code. This IDE provides an alternative for editing and running your Python code. In this lab you will be using this alternative Python runtime to create and launch your Dash applications.

Important Notice about this lab environment

Please be aware that sessions for this lab environment are not persisted. When you launch the Cloud IDE, you are presented with a 'dedicated computer on the cloud' exclusively for you. This is available to you as long as you are actively working on the labs.

Once you close your session or it is timed out due to inactivity, you are logged off, and this 'dedicated computer on the cloud' is deleted along with any files you may have created, downloaded or installed. The next time you launch this lab, a new environment is created for you.

If you finish only part of the lab and return later, you may have to start from the beginning. So, it is a good idea to plan to your time accordingly and finish your labs in a single session.

Let's start creating dash application

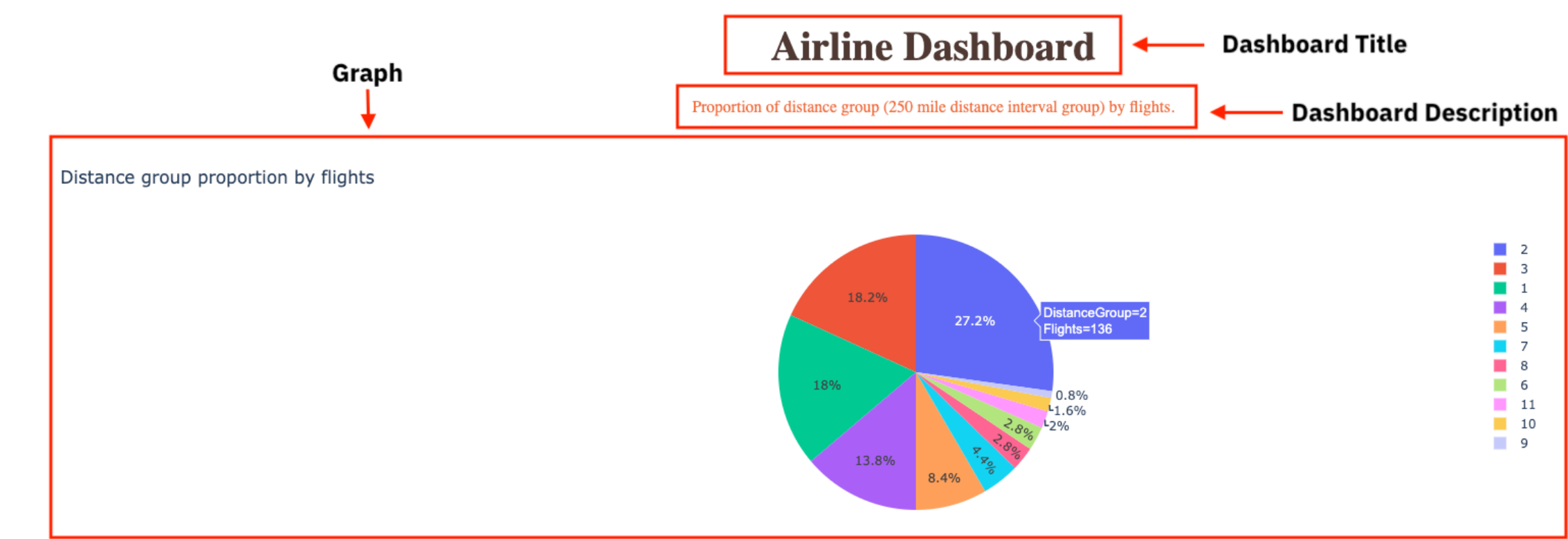
Goal

Create a dashboard that displays the percentage of flights running under specific distance group. Distance group is the distance intervals, every 250 miles, for flight segment. If the flight covers to 500 miles, it will be under distance group 2 (250 miles + 250 miles).

Expected Output

Below is the expected result from the lab. Our dashboard application consists of three components:

- Title of the application
- Description of the application
- Chart conveying the proportion of distance group by month



To do:

1. Import required libraries and read the dataset
2. Create an application layout
3. Add title to the dashboard using HTML H1 component
4. Add a paragraph about the chart using HTML P component
5. Add the pie chart above using core graph component
6. Run the app

Get the tool ready

- Install python packages required to run the application. Copy and paste the below command to the terminal.

```
python3.8 -m pip install packaging
```

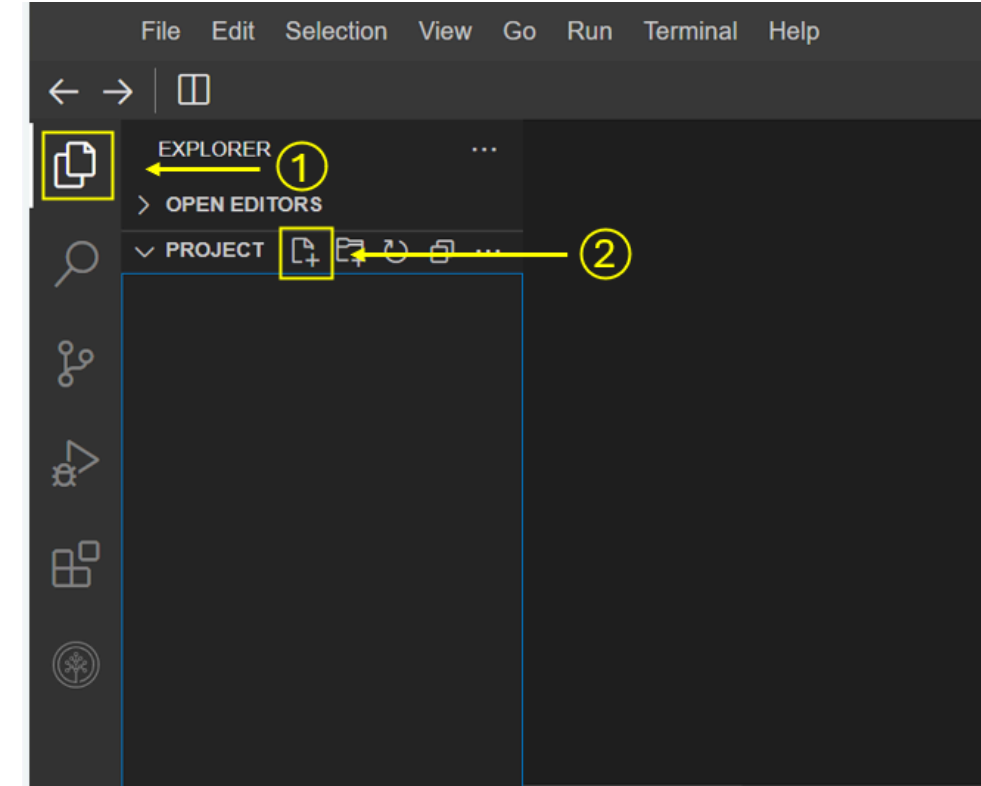
```
python3.8 -m pip install pandas dash
```

```
theia@theiadocker-malikas: /home/project ✕
theia@theiadocker-malikas:/home/projects$ python3 -m pip install pandas dash
Collecting pandas
  Downloading https://files.pythonhosted.org/packages/c3/e2/00cacacafb071c787019f00ad84ca3185952f6bb9bca9550ed83870d4d/pandas-1.1.5-cp36-cp36m-manylinux1_x86_64.whl (9.5MB)
    100% |#####| 9.5MB 163kB/s
Collecting dash
  Downloading https://files.pythonhosted.org/packages/cc/42/e1692b2d34e4135569db680efe3438e809a6b3f0ae07ad41aeff7741672/dash-2.6.1-py3-none-any.whl (9.9MB)
    100% |#####| 9.9MB 159kB/s
Collecting pytz>=2017.2 (from pandas)
  Cache entry deserialization failed, entry ignored
  Downloading https://files.pythonhosted.org/packages/d5/50/54451e88e3da4616286029a3a17fc377de817f66a0f50e1faee90161724/pytz-2022.2.1-py2.py3-none-any.whl (501kB)
    100% |#####| 501kB 3.2MB/s
Collecting python-dateutil>=2.7.3 (from pandas)
  Cache entry deserialization failed, entry ignored
  Cache entry deserialization failed, entry ignored
  Downloading https://files.pythonhosted.org/packages/36/7a/87837f39d0296e723bb9b62bbb257d0355c7f6128853c78955f57342a56d/python_dateutil-2.8.2-py2.py3-none-any.whl (247kB)
    100% |#####| 256kB 5.8MB/s
Collecting numpy>=1.15.4 (from pandas)
  Downloading https://files.pythonhosted.org/packages/45/b2/6c7545bb7a38754d63048c7696804a0d947328125d81bf12beaa692c3ae3/numpy-1.19.5-cp36-cp36m-manylinux1_x86_64.whl (13.4MB)
    100% |#####| 13.4MB 111kB/s
Collecting contextvars==2.4; python version < "3.7" (from dash)
  Downloading https://files.pythonhosted.org/packages/83/96/55b82d9f13763be9d672622e1b8106c85acb83edd7cc2fa5bc67cd9877e9/contextvars-2.4.tar.gz
Collecting dash-table==5.0.0 (from dash)
  Downloading https://files.pythonhosted.org/packages/da/ce/43f77dc8e7bbad02a9f88d07bf794eaf68359df756a28bb9f2f78e255bb1/dash_table-5.0.0-py3-none-any.whl

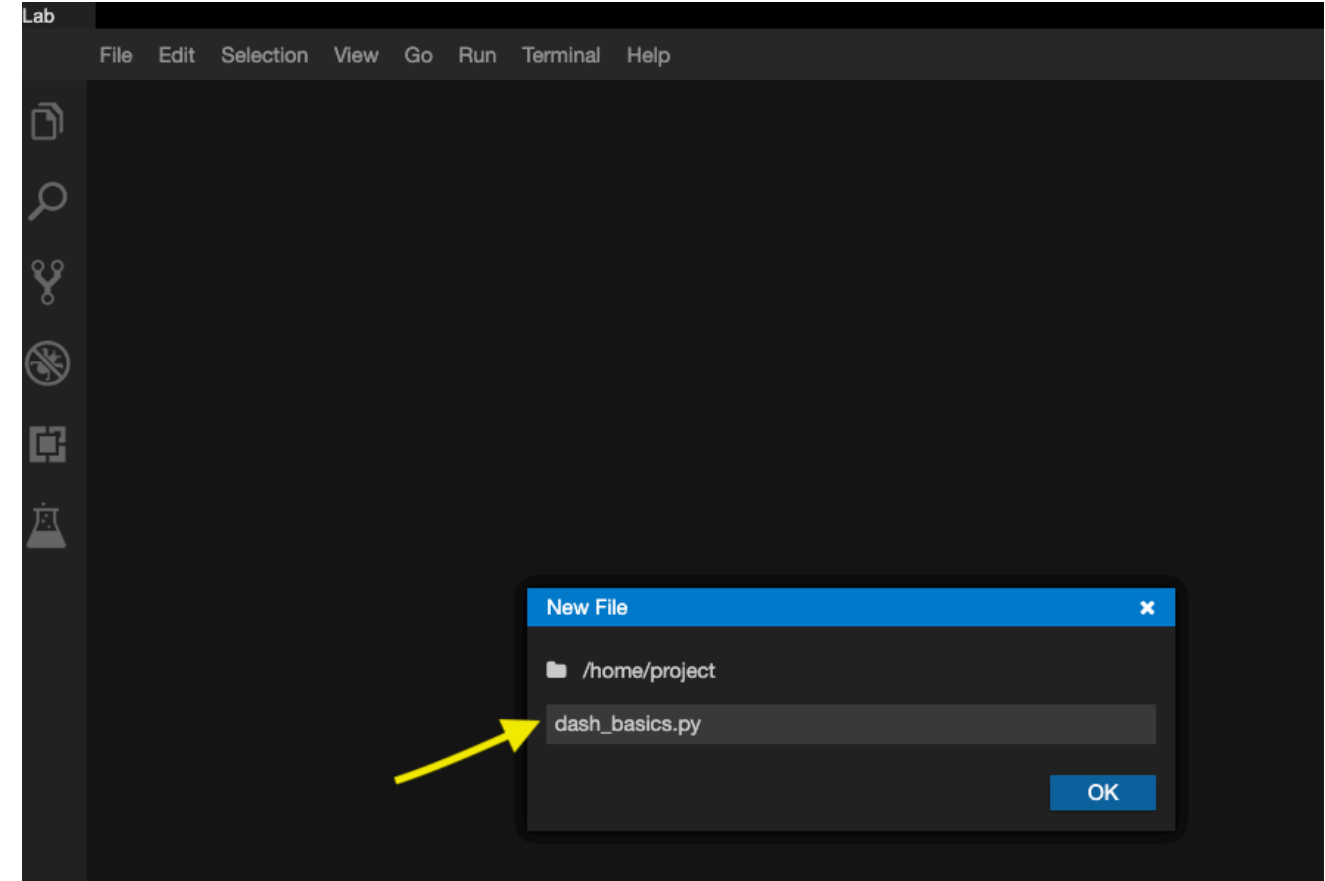
pip3 install httpx==0.20 dash plotly
```

```
theia@theiadocker-malikas: /home/project x
theia@theiadocker-malikas:/home/project$ pip3 install httpx==0.20 dash plotly
/usr/lib/python3/dist-packages/secretstorage/decrypt.py:15: CryptographyDeprecationWarning: int_from_bytes is deprecated, use int.from_bytes instead
  from cryptography.utils import int_from_bytes
/usr/lib/python3/dist-packages/secretstorage/util.py:19: CryptographyDeprecationWarning: int_from_bytes is deprecated, use int.from_bytes instead
  from cryptography.utils import int_from_bytes
Defaulting to user installation because normal site-packages is not writeable
Collecting httpx==0.20
  Downloading httpx-0.20.0-py3-none-any.whl (82 kB)
    | 82 kB 779 kB/s
Collecting dash
  Downloading dash-2.6.1-py3-none-any.whl (9.9 MB)
    | 9.9 MB 40.7 MB/s
Collecting plotly
  Downloading plotly-5.10.0-py2.py3-none-any.whl (15.2 MB)
    | 15.2 MB 39.3 MB/s
Requirement already satisfied: sniffio in /home/theia/.local/lib/python3.6/site-packages (from httpx==0.20) (1.2.0)
Requirement already satisfied: httpcore<0.14.0,>=0.13.3 in /home/theia/.local/lib/python3.6/site-packages (from httpx==0.20) (0.13.7)
Requirement already satisfied: async-generator in /home/theia/.local/lib/python3.6/site-packages (from httpx==0.20) (1.10)
Requirement already satisfied: certifi in /home/theia/.local/lib/python3.6/site-packages (from httpx==0.20) (2020.12.5)
Requirement already satisfied: rfc3986[idna2008]<2,>=1.3 in /home/theia/.local/lib/python3.6/site-packages (from httpx==0.20) (1.5.0)
Requirement already satisfied: charset-normalizer in /home/theia/.local/lib/python3.6/site-packages (from httpx==0.20) (2.0.12)
Collecting dash-html-components==2.0.0
  Downloading dash-html-components-2.0.0-py3-none-any.whl (4.1 kB)
Collecting dash-table==5.0.0
  Downloading dash-table-5.0.0-py3-none-any.whl (3.9 kB)
```

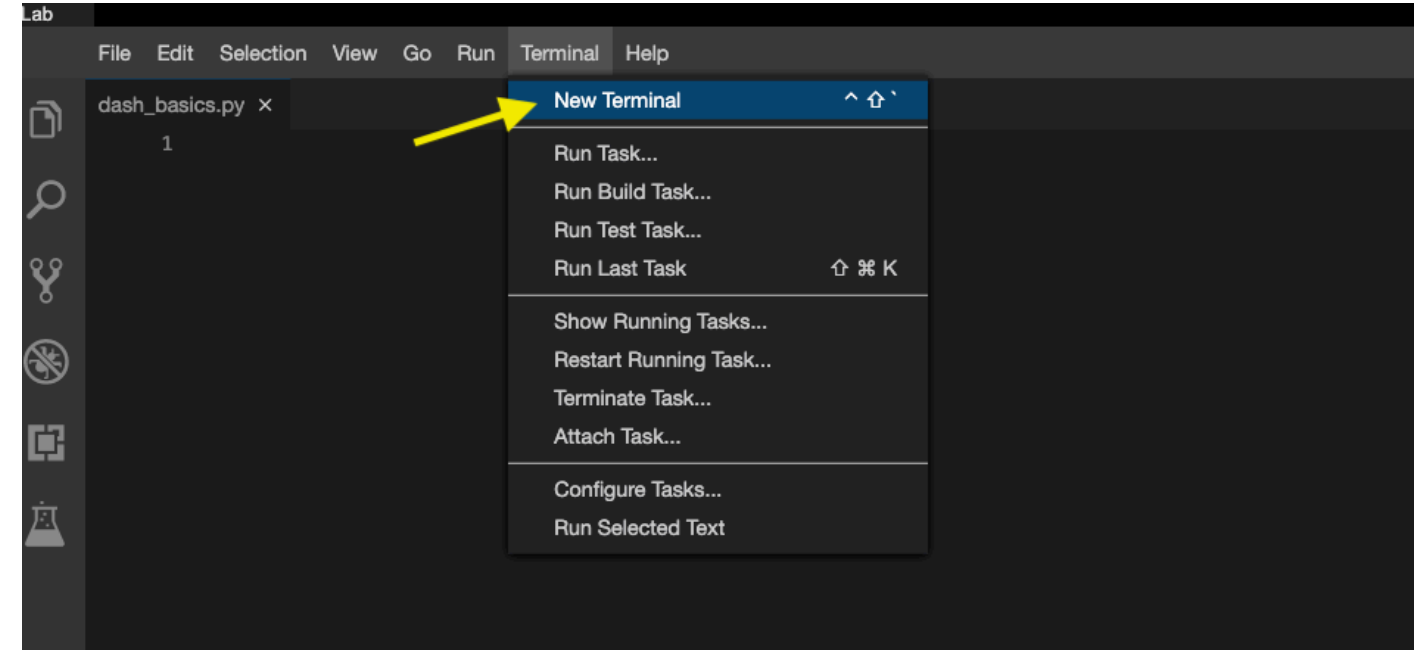
- Create a new python script, by clicking on the side tool bar **explorer** icon and selecting **new file** icon, as shown in the image below.



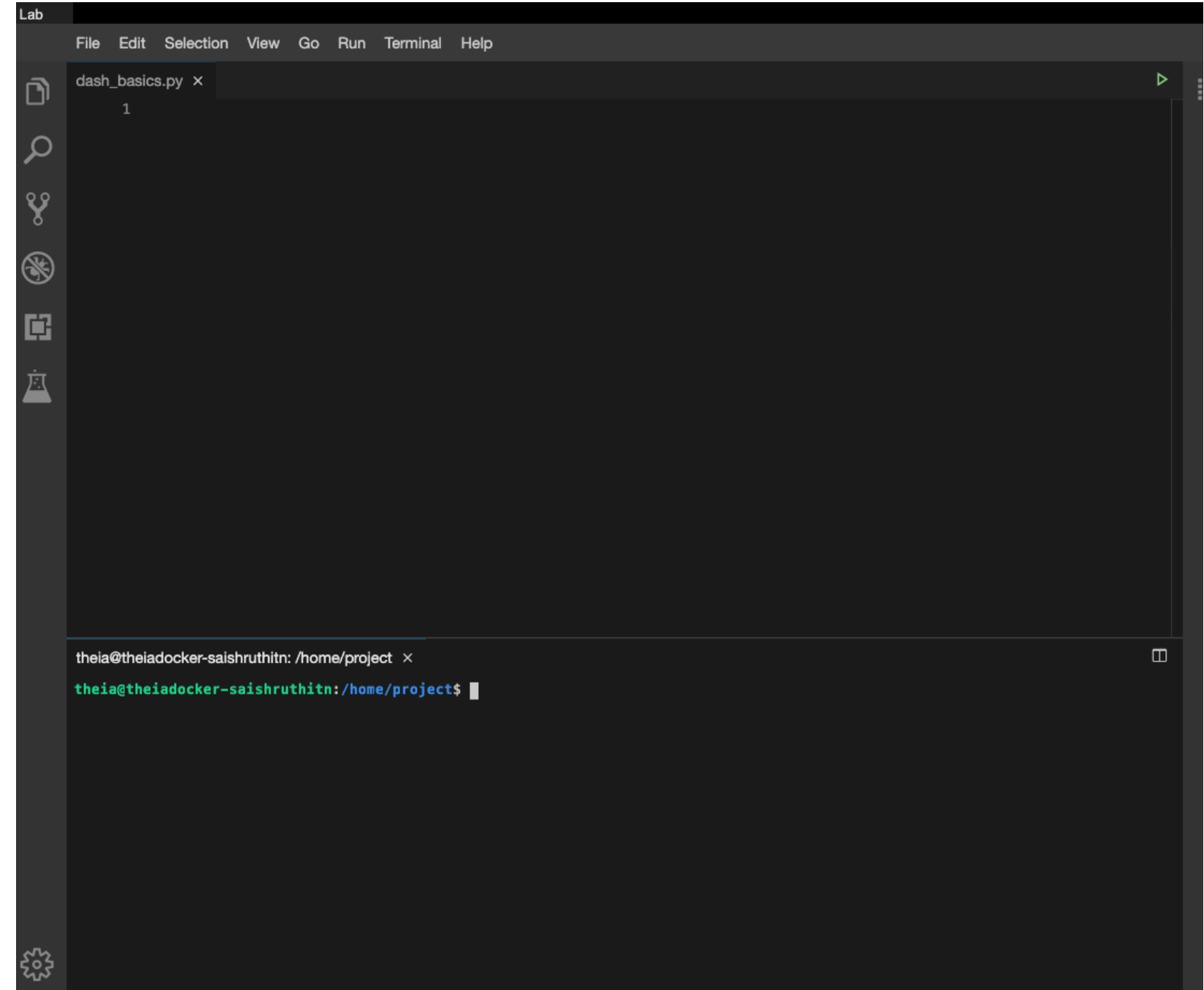
- Provide the file name as dash_basics.py



- Open a new terminal, by clicking on the menu bar and selecting **Terminal->New Terminal**, as in the image below.



- Now, you have script and terminal ready to start the lab.



TASK 1 - Data Preparation

Note: Throughout this lab, you will see placeholder text like '.....' or `groupby(.....)`. These dots should be replaced with appropriate values such as column names ('Year', 'Vehicle_Type', etc.), grouping keys, or chart titles. Refer to the comments and hints above each task to determine what needs to be filled in.

Let’s start with

- Importing necessary libraries
- Reading and sampling 500 random data points
- Get the chart ready

Copy the below code to the `dash_basics.py` script and review the code.

```
# Import required packages
import pandas as pd
import plotly.express as px
import dash
from dash import dcc
from dash import html
# Read the airline data into pandas dataframe
airline_data = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/airline_data.csv',
                           encoding = "ISO-8859-1",
                           dtype={'Div1Airport': str, 'Div1TailNum': str,
                                   'Div2Airport': str, 'Div2TailNum': str})
# Randomly sample 500 data points. Setting the random state to be 42 so that we get same result.
data = airline_data.sample(n=500, random_state=42)
# Pie Chart Creation
fig = px.pie(data, values='Flights', names='DistanceGroup', title='Distance group proportion by flights')
```

TASK 2 - Create dash application and get the layout skeleton

Next, we create a skeleton for our dash application. Our dashboard application has three components as seen before:

- Title of the application
- Description of the application
- Chart conveying the proportion of distance group by month

Mapping to the respective Dash HTML tags:

- Title added using `html.H1()` tag
- Description added using `html.P()` tag
- Chart added using `dcc.Graph()` tag

Copy the below code to the `dash_basics.py` script and review the structure.

NOTE: Copy below the current code

```
# Create a dash application
app = dash.Dash(__name__)
# Get the layout of the application and adjust it.
# Create an outer division using html.Div and add title to the dashboard using html.H1 component
# Add description about the graph using HTML P (paragraph) component
# Finally, add graph component.
app.layout = html.Div(children=[html.H1(...),
                                html.P(...),
                                dcc.Graph(...),
                                ])
# Run the application
if __name__ == '__main__':
    app.run()
```

TASK 3 - Add the application title

Update the `html.H1()` tag to hold the application title.

- Application title is Airline Dashboard
- Use style parameter provided below to make the title center aligned, with color code #503D36, and font-size as 40

```
'Airline Dashboard', style={'textAlign': 'center', 'color': '#503D36', 'font-size': 40})
```

After updating the `html.H1()` with the application title, the `app.layout` will look like:

dash_basics.py ×

```
20 # Create a dash application
21 app = dash.Dash(__name__)
22
23 # Get the layout of the application and adjust it.
24 # Create an outer division using html.Div and add title to the dashboard using html.H1 component
25 # Add description about the graph using HTML P (paragraph) component
26 # Finally, add graph component
27 app.layout = html.Div(children=[html.H1('Airline Dashboard',
28                                     style={'textAlign': 'center',
29                                     'color': '#503D36',
30                                     'font-size': 40}),
31                               html.P(),
32                               dcc.Graph(),
33                               ],)
34
35
```

TASK 4 - Add the application description

Update the `html.P()` tag to hold the description of the application.

- Description is Proportion of distance group (250 mile distance interval group) by flights.
- Use style parameter to make the description center aligned and with color #F57241.

```
'Proportion of distance group (250 mile distance interval group) by flights.', style={'textAlign': 'center', 'color': '#F57241'}
```

After updating the `html.H1()` with the application title, the `app.layout` will look like:

File Edit Selection View Go Run Terminal Help

dash_basics.py ●

```
20 # Create a dash application
21 app = dash.Dash(__name__)
22
23 # Get the layout of the application and adjust it.
24 # Create an outer division using html.Div and add title to the dashboard using html.H1 component
25 # Add description about the graph using HTML P (paragraph) component
26 # Finally, add graph component.
27 app.layout = html.Div(children=[html.H1('Airline Dashboard',
28                                     style={'textAlign': 'center',
29                                     'color': '#503D36',
30                                     'font-size': 40}),
31                               html.P('Proportion of distance group (250 mile distance interval group) by flights.',
32                                     style={'textAlign': 'center', 'color': '#F57241'}),
33                               dcc.Graph(),
34                               ],)
35
36
```

TASK 5 - Update the graph

Update `figure` parameter of `dcc.Graph()` component to add the pie chart. We have created pie chart and assigned it to `fig`. Let's use that to update the `figure` parameter.

```
figure=fig
```

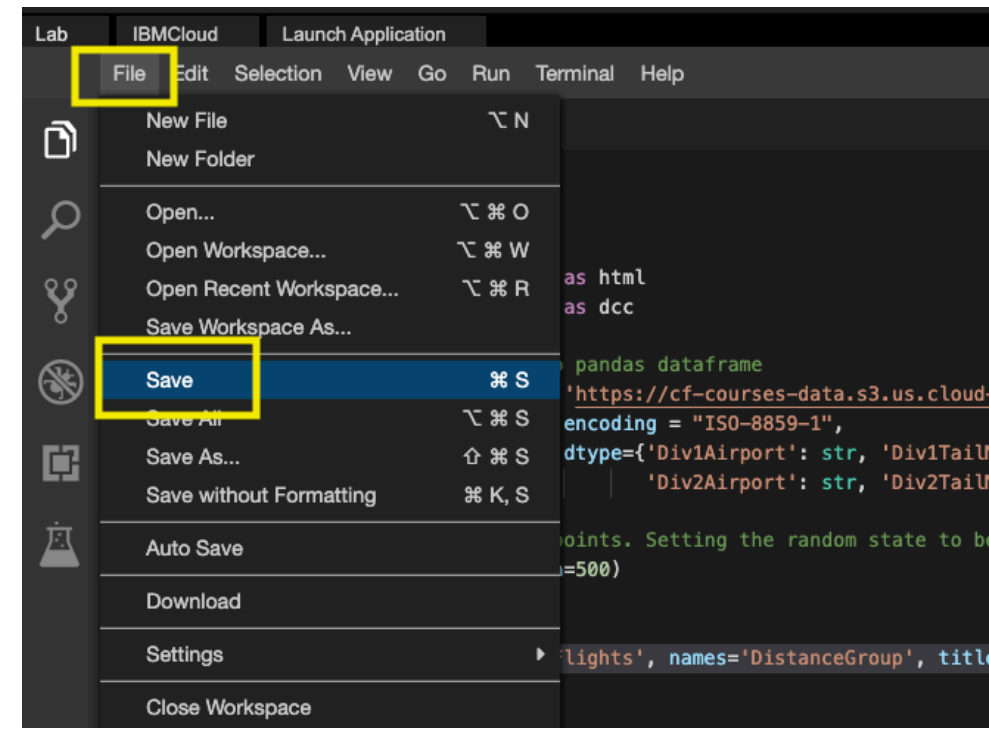
After updating the `dcc.Graph()` with the application title, the `app.layout` will look like:

File Edit Selection View Go Run Terminal Help

dash_basics.py ×

```
20 # Create a dash application
21 app = dash.Dash(__name__)
22
23 # Get the layout of the application and adjust it.
24 # Create an outer division using html.Div and add title to the dashboard using html.H1 component
25 # Add description about the graph using HTML P (paragraph) component
26 # Finally, add graph component.
27 app.layout = html.Div(children=[html.H1('Airline Dashboard',
28                                     style={'textAlign': 'center',
29                                     'color': '#503D36',
30                                     'font-size': 40}),
31                               html.P('Proportion of distance group (250 mile distance interval group) by flights.',
32                                     style={'textAlign': 'center', 'color': '#F57241'}),
33                               dcc.Graph(figure=fig),
34
35                               ])
36
```

Before running the application, save the file by clicking on **File -> Save** from the menu bar.



You can Refer to the entire python code here

```
# Import required packages
import pandas as pd
import plotly.express as px
import dash
from dash import dcc
from dash import html
# Read the airline data into pandas dataframe
airline_data = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/airline_data.csv',
                           encoding = "ISO-8859-1",
                           dtype={'Div1Airport': str, 'Div1TailNum': str,
                                   'Div2Airport': str, 'Div2TailNum': str})

# Randomly sample 500 data points. Setting the random state to be 42 so that we get same result.
data = airline_data.sample(n=500, random_state=42)
# Pie Chart Creation
fig = px.pie(data, values='Flights', names='DistanceGroup', title='Distance group proportion by flights')
# Create a dash application
app = dash.Dash(__name__)
# Get the layout of the application and adjust it.
# Create an outer division using html.Div and add title to the dashboard using html.H1 component
# Add description about the graph using HTML P (paragraph) component
# Finally, add graph component.
app.layout = html.Div(children=[html.H1('Airline Dashboard', style={'textAlign': 'center', 'color': '#503D36', 'font-size': 40}),
                               html.P('Proportion of distance group (250 mile distance interval group) by flights.', style={'textAlign': 'center', 'color': '#F57241'}),
                               dcc.Graph(figure=fig),

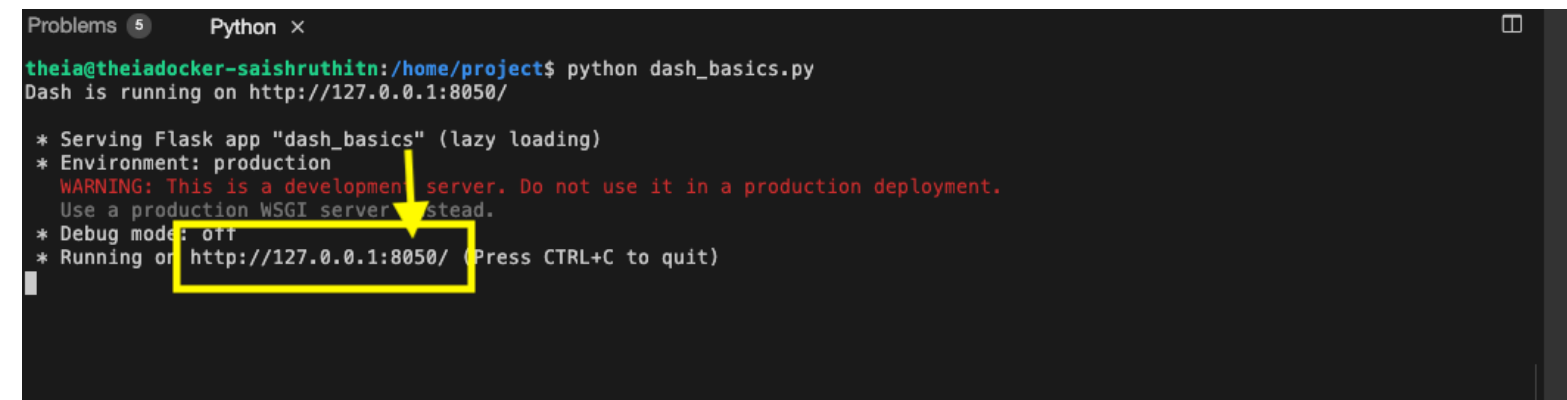
                               ])

# Run the application
if __name__ == '__main__':
    app.run()
```

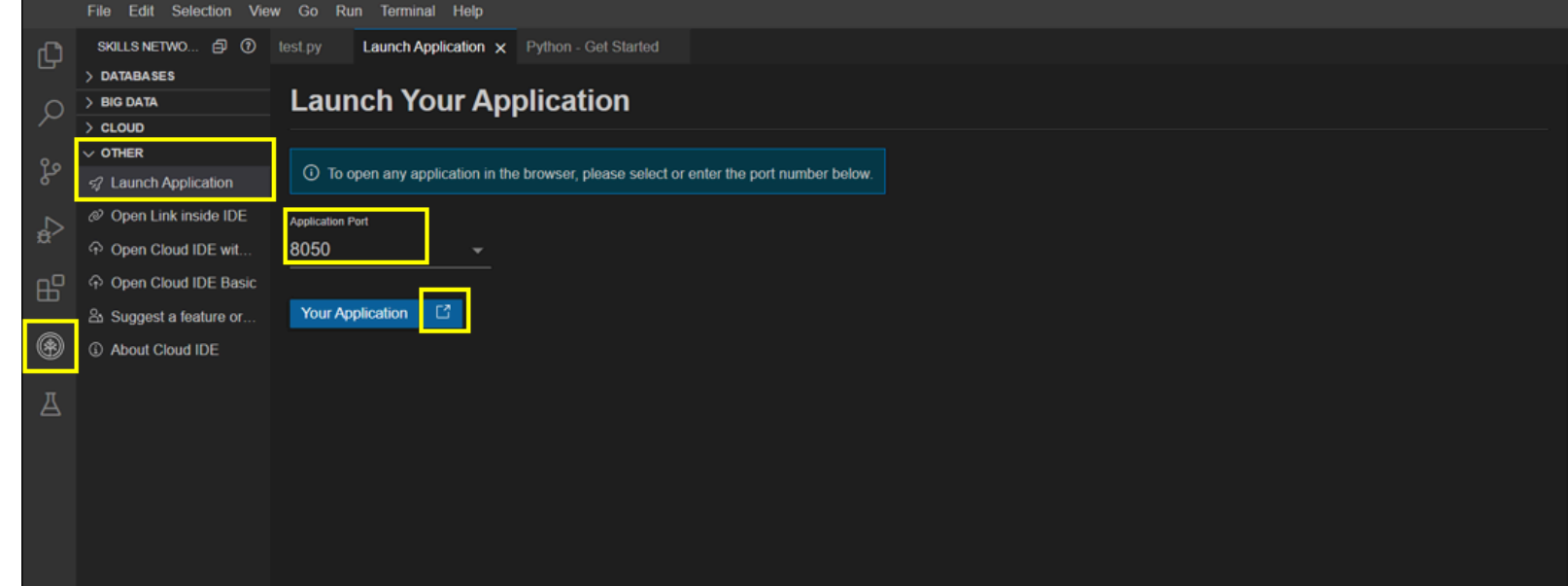
TASK 6 - Run the application

- Run the python file using the following command in the terminal
- ```
python3.8 dash_basics.py
```

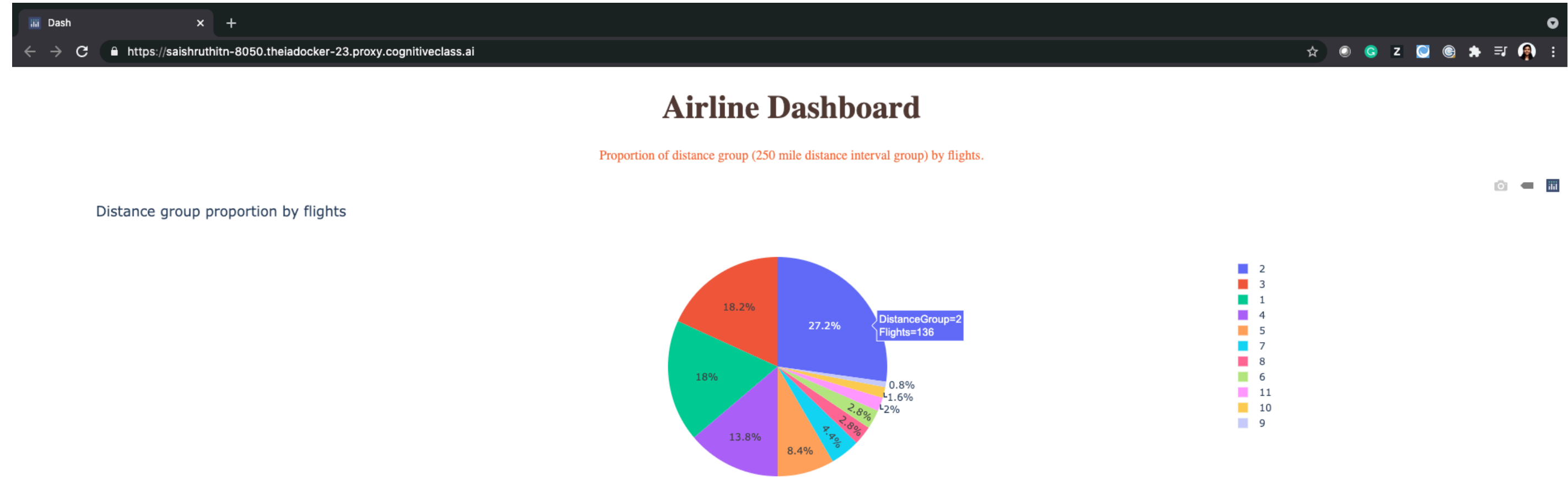
- Observe the port number shown in the terminal.



- Click on the Launch Application option from the side menu bar. Provide the port number and click OK



The app will open in a new browser tab like below:



Congratulations, you have successfully created your first dash application!

## Exercise : Practice Tasks

You will practice some tasks to update the dashboard.

- Change the title to the dashboard from “Airline Dashboard” to “Airline On-time Performance Dashboard” using HTML H1 component and font-size as 50.

▼ Answer

html.H1(‘Airline On-time Performance Dashboard’,style={‘textAlign’: ‘center’, ‘color’: ‘#503D36’, ‘font-size’: 50}),
- Save the above changes and relaunch the dashboard application to see the updated dashboard title.

▼ Answer

Click on file -> save file.Then go to terminal and Run the command python3 dash\_basics.py to open the updated file again and relaunch the application by entering the port number.The updated dashboard title will be seen.
- Write a command to stop the running app in the terminal

▼ Answer

Press ctrl+c inside the terminal to stop the dash application.

## Author

[Saishruthi Swaminathan](#)

© IBM Corporation. All rights reserved.